ARGUS Subsystems

How the system fits together

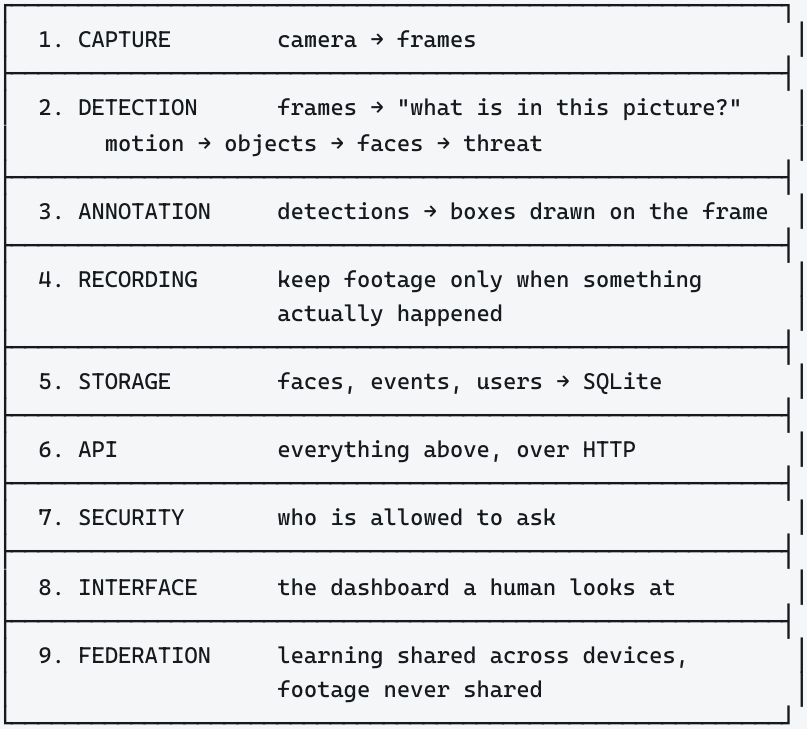
ARGUS Subsystems

The whole system in plain terms, one layer at a time. No prior knowledge of the codebase assumed.

The one-sentence version

A camera watches a scene; a chain of increasingly expensive AI models decides whether anything worth caring about is in the frame; the answer is drawn on a live video feed and saved to disk; and every two weeks the device sends what it learned — never what it saw — to a central server that combines it with other devices.

The nine subsystems



1. Capture — getting pictures off the camera

Job: turn a physical camera into a stream of images the rest of the system can use.

Three different kinds of computer need three different ways of talking to a camera, and the wrong one fails in confusing ways:

Board	How it talks to the camera
Raspberry Pi	`picamera2`
Jetson Orin	GStreamer (`nvarguscamerasrc`)
Laptop / dev machine	OpenCV

The system works out which board it is on by reading the board's own ID from the operating system. This sounds trivial and is not: the original code guessed from the processor type, and since a Jetson and a Pi both use ARM chips, it would confidently pick the Pi method on a Jetson and report a camera failure that was really a software bug.

Several cameras are supported at once, each one identified by which connector it is plugged into.

> **Files:** `src/camera/` — `platform_detect.py` , `backends.py` , `__init__.py`

---

2. Detection — deciding what is in the picture

Job: look at a frame and say what is in it.

This is the heart of the project, and its design principle is simple: **cheap checks first, expensive checks only when the cheap ones say it is worth it.** Running every model on every frame would be far too slow.

```
every frame ──► Is anything moving?          (very cheap)
                      │ yes
                      ▼
                What objects are here?        (moderate — AI model)
                      │ found a person
                      ▼
                Who is this person?           (expensive — AI model)
                Is anyone armed?              (moderate — AI model)
```

If nothing moves, nothing else runs. A still room costs almost nothing.

The seven things it can report

Class	Meaning	Comes from
`MOTION`	Something changed between frames	Comparing consecutive images
`HUMAN`	A person	YOLOv8 object model
`VEHICLE`	Car, truck, bus, motorcycle	YOLOv8 object model
`ANIMAL`	Cat, dog, horse, sheep, cow	YOLOv8 object model
`UNKNOWN`	An object it recognises but we do not categorise	YOLOv8 object model
`FACE`	A face, with a name if enrolled	ArcFace face model
`DANGEROUS_PERSON`	A person appearing with a weapon-like object	YOLO11 threat model

Three separate AI models are involved. They do not share a brain — each answers one question, and the system combines the answers.

> **Files:** `src/detection/` — `__init__.py`, `face_recognition.py`, `threat.py`

---

3. Annotation — drawing the answer on the picture

Job: burn boxes and labels into the video so a human can see what the system decided.

Colour	Meaning
Orange	Person flagged as carrying a weapon
Green	Recognised person, or any ordinary object
Red	A face the system does not recognise

Boxes are drawn on the server, before the video is sent, so a label can never drift out of sync with the frame it describes.

One subtlety: a flagged person is detected twice — once as a person, once as a threat. Drawing both stacks two labels on the same spot and implies two people are there. The threat box replaces the person box.

> **Files:** `src/detection/annotate.py`

---

4. Recording — keeping only what matters

Job: cameras run 24/7; hard drives do not.

Recording everything fills a memory card in hours and buries the ten seconds anyone actually wants. So footage is saved **only around real detections**, with three refinements that make the clips usable:

- **Pre-roll.** The clip starts *before* the detection. A clip that begins the instant someone is spotted has already missed them walking up, which is usually the useful part.
- **Post-roll.** Recording continues briefly after, so someone walking out of frame does not get cut off mid-stride.
- **Cooldown.** Someone standing around produces *one* clip, not three hundred nearly identical ones.

Motion alone deliberately does not trigger recording — it fires on clouds and moving branches, and recording on that is 24/7 recording with extra steps.

> Files: `src/detection/event_recorder.py`

---

5. Storage — remembering things

Job: keep what needs to outlive a restart.

A single SQLite database file holds enrolled faces (as mathematical descriptions, not photographs), an event log, user accounts, and an audit trail. Video clips are files on disk alongside it.

> Files: `src/database/__init__.py`

---

6. API — the way everything is asked

Job: expose all of the above over HTTP so anything can talk to it — the dashboard, a phone, a script, a teammate's app.

38 endpoints across camera control, detection status, face enrolment, events, recordings, federated learning, and authentication.

> Files: `src/api/routes.py`, `src/api/app.py`

---

7. Security — deciding who may ask

Job: make sure not just anyone can watch the camera.

Before this existed, every endpoint was open — including the live video feed and the ability to delete enrolled faces. Login was a stub that returned "Authentication not implemented".

Now: log in with a username and password, receive a token, and send that token with every request. Passwords are stored scrambled (hashed with a random salt), never as text.

Authentication is **off by default**, because the demo runs on a private network where it would just be friction. It becomes **mandatory** the moment the system is exposed to the internet — and the script that opens that tunnel refuses to run without it, and proves auth is on by checking that an unauthenticated request is actually rejected.

> Files: `src/api/auth.py`, `src/security/__init__.py`

---

8. Interface — what a person looks at

Job: one page showing the live feed and the controls.

Live annotated video, status badges (frame rate, which accelerator is active, which models loaded), enrol a face, list and remove faces, reset between visitors, and a threat banner.

Deliberately a single file with no build step and **no external downloads** — the demo unit runs offline, and a web font or script loaded from the internet would work perfectly on a laptop and fail silently at the venue.

> Files: `static/index.html`

---

9. Federation — learning together without sharing footage

Job: let many devices improve one shared model without any of them handing over their video.

This is the part that makes the project research rather than assembly.

The ordinary way to improve an AI model is to collect everyone's data in one place and train on it. For security cameras that means shipping footage of people's homes to a company server.

Federated learning inverts it. The model travels to the data instead:

```
Day 0    Every device has the same model.
Days 1–13   Each device learns from what its own camera saw.
            Nothing is transmitted.
Day 14   Each device sends only the MODEL CHANGES — a list of
         numbers. No images, ever.
Server   Combines the changes into a new shared model and sends
         it back to everyone.
```

The part that needed real engineering

Combining changes by simple averaging has an obvious flaw: **one broken or malicious device poisons everyone.** Its bad numbers get averaged in, and the damaged model is sent back to every device.

So updates are screened before averaging:

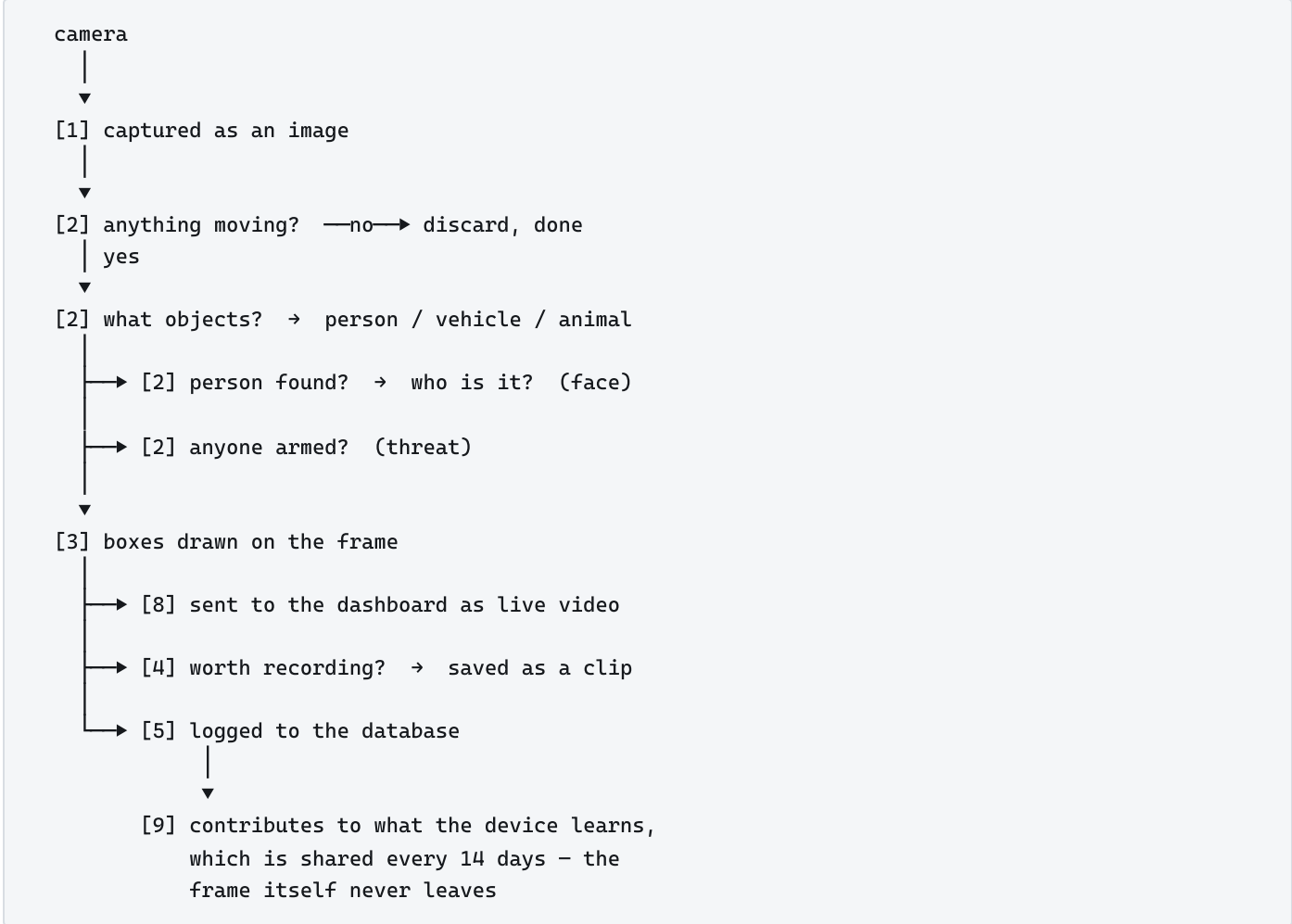
1. **Structural** — reject anything malformed or containing invalid numbers. One invalid value, averaged in, destroys the model entirely.
2. **Size** — reject updates enormously larger than everyone else's, the signature of a device trying to overwrite the model.
3. **Direction** — reject updates pointing the opposite way to everyone else's, the signature of deliberate sabotage.
4. **Trimming** — discard the most extreme values before averaging.

Then a final check: the new model is tested, and if it performs worse than the one it would replace, **it is thrown away and the old model kept**.

An honest limitation: steps 3 and 4 compare devices against each other, so they need at least **three** contributors to mean anything. With two devices that disagree, there is no way to know which is wrong. The system detects this, reports it, and falls back to the checks that work regardless.

> Files: `src/federated/` — `robust.py`, `scheduler.py`, `client.py`

How a single frame travels through all of it



Where the time goes

Not all subsystems cost the same. Measured on a desktop CPU:

Stage	Cost	Runs when
Motion	negligible	Always
Objects (YOLO)	~90 ms	Something moved
Threat (YOLO11)	~36 ms	Something moved
Faces (ArcFace)	expensive	A person was found

On a Raspberry Pi with a Hailo accelerator, the two YOLO stages move onto dedicated hardware and become very fast. **Face recognition does not** — it stays on the main processor. So on the final hardware, face recognition is the bottleneck, not object detection. That is a counter-intuitive result and it decides where optimisation effort

goes.

What each subsystem is worth in code

Subsystem	Lines	Files
Detection (all four stages)	1,960	8
Federation	1,049	6
API + auth	1,095	4
Capture	850	3
Training	552	2
Storage	286	1
Security primitives	222	1
Interface	502	1
Tooling scripts	1,183	7
Tests	**2,959**	**15**

249 automated tests, all passing. Tests are the largest single body of code in the project, which is deliberate: on a demo day, the failures that matter are the silent ones.